agent-runtime

Deep Dive: an event-sourced runtime for tool-using LLM agents

A beginner-safe, exhaustive walkthrough of the actual codebase

How to read this document

This document explains exactly what is implemented in the `agent-runtime` project folder. Every claim traces to a specific file, and where a claim could be checked by running the code, it was run: the test suite, the command line tool, the scenario suite and the replay path were all executed while writing this, and what they printed is reported here rather than assumed. Technical terms are defined the first time they appear, in a boxed callout. Assume no prior knowledge: if you have never met event sourcing or deterministic replay, Sections 2 and 9 build both from nothing.

Red boxes are honest findings. Some repeat limitations the project's own README already states; others are defects found by reading and running the code that the README does not mention. The two are labelled differently, because the second kind is the kind an interviewer might find first.

Contents

1 What this project is

agent-runtime is a Python library and command line tool for running *LLM agents* with the operational machinery that a bare "call the model, run a tool, repeat" loop leaves out: a durable record of every run, the ability to replay a run exactly, recovery from a crash mid-run, a human approval step before anything with a side effect happens, deliberately injected failures to prove the system degrades safely, and a suite of scripted regression tests that catch behavior changes when the underlying model changes.

Jargon: LLM agent

A *large language model* (LLM) is a machine learning model that produces text from text. An *LLM agent* is a program that asks such a model, in a loop, "given what has happened so far, what should happen next?" The model's answer is either a final text answer, or a request to call one or more *tools*: functions the program exposes, such as "read this file" or "send this HTTP request". The program executes the requested tools, feeds the results back to the model, and asks again. This project implements exactly that loop in `src/agent_runtime/runtime.py`.

1.1 Why it exists, in the project's own framing

The README states the thesis directly: the agent loop itself is a small amount of code, but what makes an agent *deployable* is everything around it. Knowing exactly what happened in a specific run. Being able to reproduce that run. Knowing whether the agent emailed a customer before a human approved it. Knowing whether a new model version still passes the flows that used to work.

The design response to all of this is a single idea, applied consistently: **every run is an append-only log of events, and every feature is a function computed from that log.**

2 Event sourcing, from first principles

Almost every program you have used stores *current state*. A spreadsheet holds the value in each cell. When you change a cell, the old value is overwritten and gone. This is simple and it is what most software does.

Event sourcing inverts that.

Jargon: Event sourcing

A design pattern where, instead of storing "the current state" of something and overwriting it as things change, you store every state-changing *event* as an immutable, ordered record, and compute "current state" on demand by replaying those events from the start. The log never loses information, so you can always recompute the past, reproduce it, or audit it.

A bank account is the classic illustration. You could store one number, the balance, and overwrite it on every transaction. Or you could store the list of deposits and withdrawals, and compute the balance by adding them up. The second is slower to read and it uses more disk, and in exchange it can answer questions the first can never answer: what was my balance on the 3rd? Why is it this number? Which transaction was wrong? Undo that one thing without losing the rest.

Why an AI agent in particular wants this

An agent's problems are exactly the problems the log solves.
An agent is *nondeterministic*: ask the same model the same question twice and you may get different answers. It is *expensive*: every step costs money and seconds. It *acts on the world*: it writes files and drafts emails, and those actions do not un-happen. And it is *opaque*: when it does something strange, the only honest answer to "why did it do that?" is a complete record of what it saw and what it chose.
Store only current state and you have none of that. You have a finished run, a changed workspace, and no way to answer any question about how you got there. Store the log and "what happened", "do it again exactly", "continue from the crash" and "did the new model change behavior" all become the same kind of question: a function over a list of events.

The rest of this document is, in a real sense, that one idea and its consequences. Every feature in Sections 7 through 15 is a function over the log.

3 Architecture at a glance

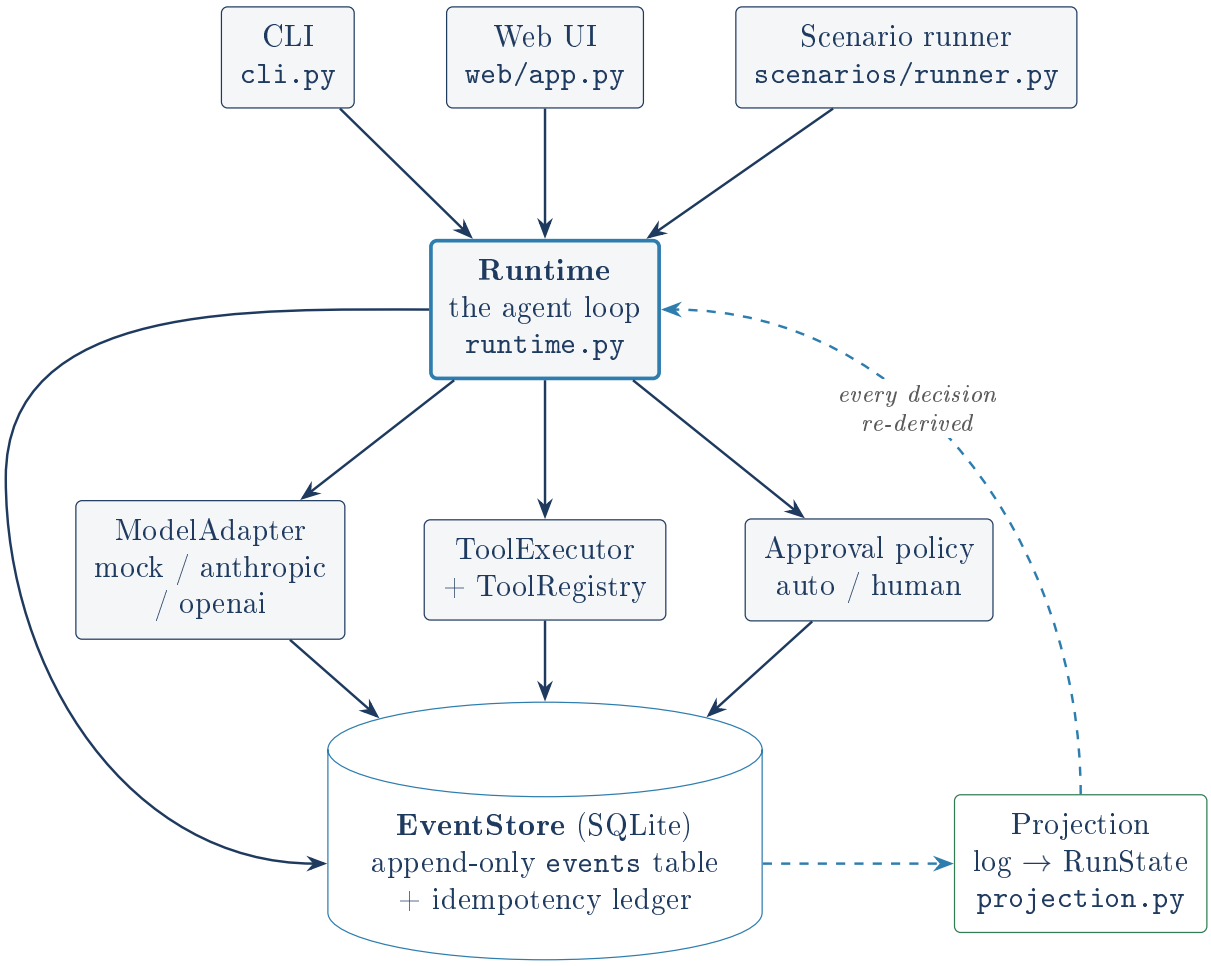

Figure 1: Solid arrows are appends; dashed arrows are reads through the projection. Nothing reads the store except through `project()`.

Every arrow into the SQLite store is an *append*. The codebase contains no UPDATE and no DELETE against the `events` table: the schema and every SQL statement in `store.py` is an `INSERT` or a `SELECT`, confirmed by reading that file in full. Every arrow out of the store goes through

`projection.py`, the only module that interprets what a sequence of events means. The CLI, the web UI, the replay comparator and the scenario assertions all call the same `project()` function, so they cannot disagree about the state of a run.

Jargon: SQLite

A small relational database engine that needs no separate server process: the whole database is one ordinary file on disk. This project uses it as the single persistent store for every run's event log, and separately as the storage behind the example calendar tool.

4 The event log

4.1 The table

From `store.py`, the schema actually created is:

```
CREATE TABLE IF NOT EXISTS events (
    run_id  TEXT    NOT NULL,
    seq     INTEGER NOT NULL,
    type    TEXT    NOT NULL,
    ts      REAL    NOT NULL,
    payload TEXT    NOT NULL,   -- a JSON blob
    PRIMARY KEY (run_id, seq)
);
CREATE INDEX IF NOT EXISTS idx_events_type ON events (run_id, type);
CREATE TABLE IF NOT EXISTS executions (      -- the idempotency ledger, Sec. 8
    idem_key TEXT PRIMARY KEY,
    run_id   TEXT NOT NULL,
    result   TEXT NOT NULL,
    ts       REAL NOT NULL
);
```

Jargon: Primary key, and what append-only means here

A *primary key* is a column or combination of columns that must be unique across rows. Here it is `(run_id, seq)`: within one run, each sequence number can be claimed exactly once. `EventStore.append()` reads `MAX(seq)+1` for the run, tries to insert, and retries (up to 5 times) if another writer claimed that number first. SQLite's uniqueness constraint rejects the loser, who re-reads the new maximum and tries again. That is how two processes can append to one run safely without a lock.

4.2 The twelve event types

`events.py` defines twelve event type constants. Grouped in the order a typical successful run emits them:

Event type	Payload highlights
`run_created`	agent name, the user's request text, and free-form `meta`: workspace path, which adapter, scenario file and behavior version if any
`model_requested`	a call number, plus a *digest* and count of the messages about to be sent, not the messages themselves
`model_responded`	the parsed turn (text and/or tool calls), token usage, model name; or `malformed: true` with the raw unparseable text
`model_error`	a transport or provider failure for one attempt, such as an HTTP 500
`tool_requested`	call id, tool name, arguments, and whether the tool is side-effecting
`tool_approved` `tool_denied`	who decided, and the deny reason if any
`tool_executed` `tool_failed`	the tool's result or error, the attempt count, and a `replayed` flag
`checkpoint`	how many model calls and resolved tool calls have happened so far
`run_finished` `run_failed`	the model's final answer text, or a machine-readable failure cause

Jargon: Digest, or hash

A short fixed-length string computed from a larger piece of data, such that the same input always gives the same digest and different inputs almost never collide. This project uses SHA-256 truncated to 16 hex characters purely as a fingerprint of the outgoing message list (`runtime._digest`). It is not a secret, just a compact "was this the same request" marker. Storing the digest rather than the messages keeps the log small, and the messages are recoverable anyway: they are a pure function of the log (Section 6).

4.3 A real log, end to end

This is not a sketch. It is the event sequence produced by actually running `agentrun run --scenario scenarios/ops_assistant/chase_overdue_acme.yaml`, approving the gated call, and reading the log back with `agentrun runs show`.

Figure 2: The 15 events of run `5eae35be7bc7`, verified by running it. Without an auto-approve policy the loop returns `waiting_approval` after seq 8, and seq 9 is written later by `agentrun approvals approve`. Events 10 to 14 are identical either way, because they are computed from the log, not from which code path produced the approval.

4.4 Canonical encoding, for exact comparison

Every `Event` has a `canonical()` method that JSON-encodes it with sorted keys and no whitespace:

```
1 json.dumps(body, sort_keys=True, separators=(",", ":"))
```

Two events that mean the same thing always produce the exact same string. This is what lets replay (Section 9) claim "byte-identical" rather than "looks about right". Note what `canonical()` excludes by default: the `run_id`. A replayed run has a different id by construction, so comparing logs would fail immediately if the id were included. Everything else, including the timestamp, must match exactly.

5 The loop: `Runtime.drive()`

`runtime.py` is the heart of the project, and `drive(run_id)` is, in the project's own words, "a fixed point iteration over the log".

Jargon: Fixed point iteration

A loop that repeatedly applies the same rule to its own output until the output stops changing or a stated exit condition is hit. Here the rule is: derive the current state from the log, decide the single next thing to do, append it, repeat. It is applied identically whether this is the first iteration of a brand new run, the continuation of a run a human just approved, or the resumption of a run that crashed a minute ago. There is one code path, not three.

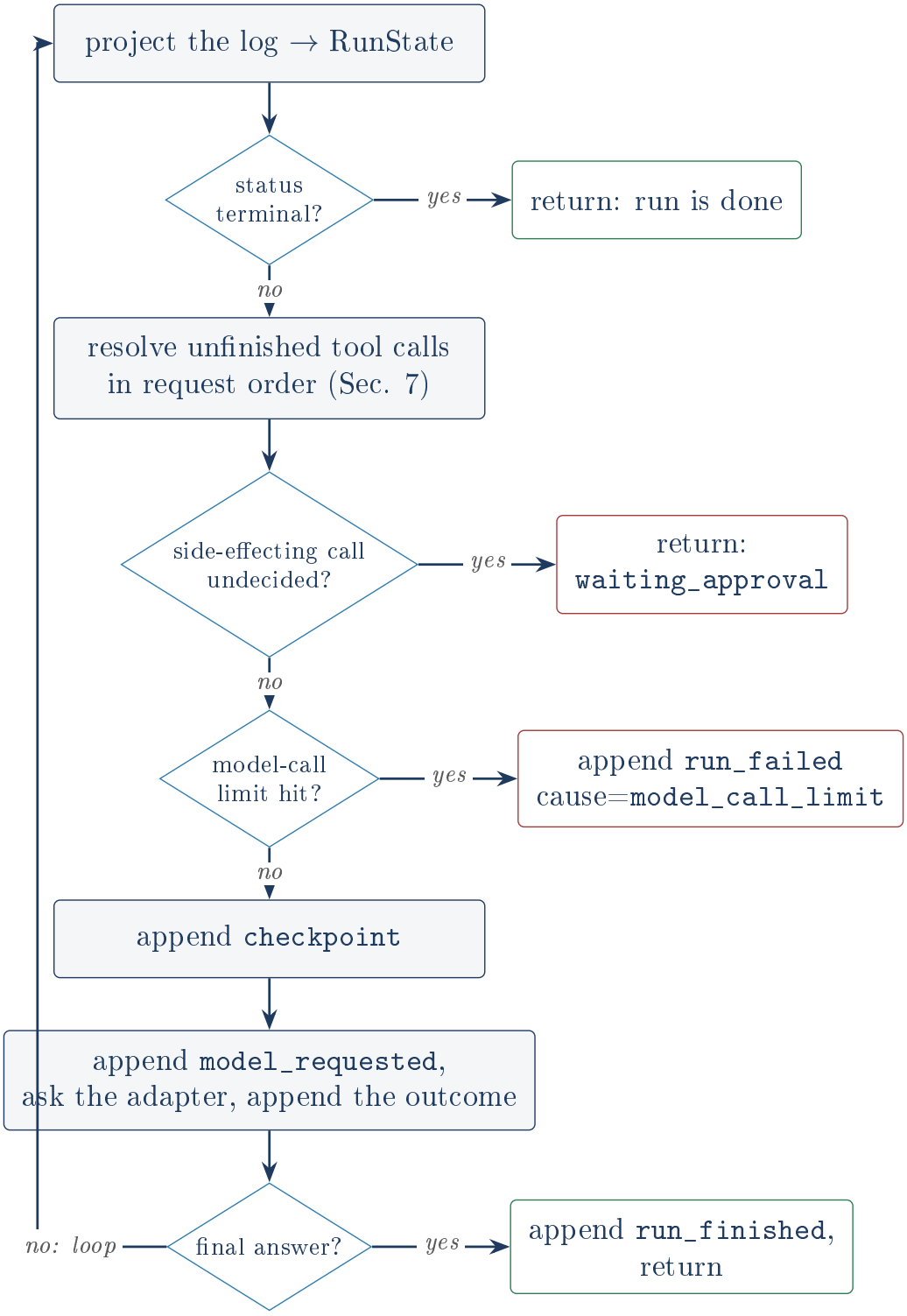

Figure 3: `Runtime.drive()`. Every exit is an append; every entry re-reads the log.

5.1 Asking the model

`_model_step()` rebuilds the conversation from the log via `build_messages()` (Section 6), appends `model_requested`, then calls `adapter.complete(messages, tool_specs)`. Three outcomes are handled:

1. **Transport or provider failure** (`AdapterError`): append `model_error` and, if retries remain (default 2), sleep $0.5 \times 2^{\text{attempt}}$ seconds before retrying. If retries are exhausted, append `run_failed` with cause `adapter_error: retries exhausted`.

2. **Unparseable output** (`MalformedOutputError`): append `model_responded` with `malformed: true` and the raw text, truncated to 2000 characters. If this is more than `max_malformed` (default 2) malformed responses in the run, fail the run with cause `malformed_model_output`. Otherwise return, so the next iteration feeds a corrective message back.
3. **A parsed turn**: append `model_responded`. If the turn has no tool calls (`ModelTurn.is_final`), it is the final answer and `run_finished` is appended. Otherwise each requested tool call becomes a `tool_requested` event, tagged with whether that tool is side-effecting, looked up from the registry.

Jargon: Exponential backoff

A retry strategy where each successive retry waits longer than the last (here doubling: 0.5s, then 1s, then 2s), so a struggling remote service is not immediately hammered again at full speed.

5.2 Checkpoints

Before each model call except the first, the loop appends a `checkpoint` recording how many model calls and resolved tool calls have happened. Read literally, a checkpoint is *not* required for correctness: every number in it is already recomputable from the rest of the log. It is a cheap progress marker for a human skimming `agentrun runs show`.

The checkpoint is a missed opportunity, and the README says so obliquely

The README's own "what I'd do differently" says resume should record "the adapter cursor in a checkpoint payload instead of inferring it". The `checkpoint` event is already being written on every iteration and already carries two counters that are pure functions of the log, which is to say it carries nothing that is not already known. The one number that is *not* recoverable from the log, the mock adapter's script position, is the one it does not record. Section 18 shows this is not hypothetical: it is a live off-by-one bug.

6 Rebuilding the conversation from the log

`projection.build_messages(state, system_prompt)` turns the log into the message list a model API expects. It walks the log in order and translates:

Log event	Message produced
`model_responded` (normal)	an `assistant` message: the turn's text, plus `tool_calls` if any
`model_responded` (malformed)	a `user` message: "Your previous output could not be parsed. Reply with either tool calls or a final answer." This is the corrective nudge from Section 5.2
`tool_executed`	a `tool` message carrying the result
`tool_failed`	a `tool` message: `{"error": ...}`
`tool_denied`	a `tool` message: `{"denied": true, "reason": ...}`

The last row is worth pausing on. It is literally how "the operator said no" becomes something the model can react to. A denial is not an exception and not a stop; it is an observation, in the same channel a tool result would arrive on. The model reads it and chooses what to do next, exactly as it would after any other tool outcome.

Because this is a pure function of the log, a run resumed after a crash sends the model exactly the messages an uninterrupted run would have sent. There is no separate "resume state" to keep in sync, because there is no state at all.

7 The approval gate

Jargon: Side effect

An action with a lasting, observable consequence outside the running program: writing a file, updating a spreadsheet, adding a calendar entry, drafting an email. Contrast a *read-only* action (reading a file, listing a directory), which can be safely retried because it changes nothing. Every tool in this project declares a boolean `side_effect` flag; only side-effecting tools are gated.

`RunState.unfinished_calls()` returns every requested tool call still `requested` or `approved`, in request order. `_process_tool_calls()` walks them:

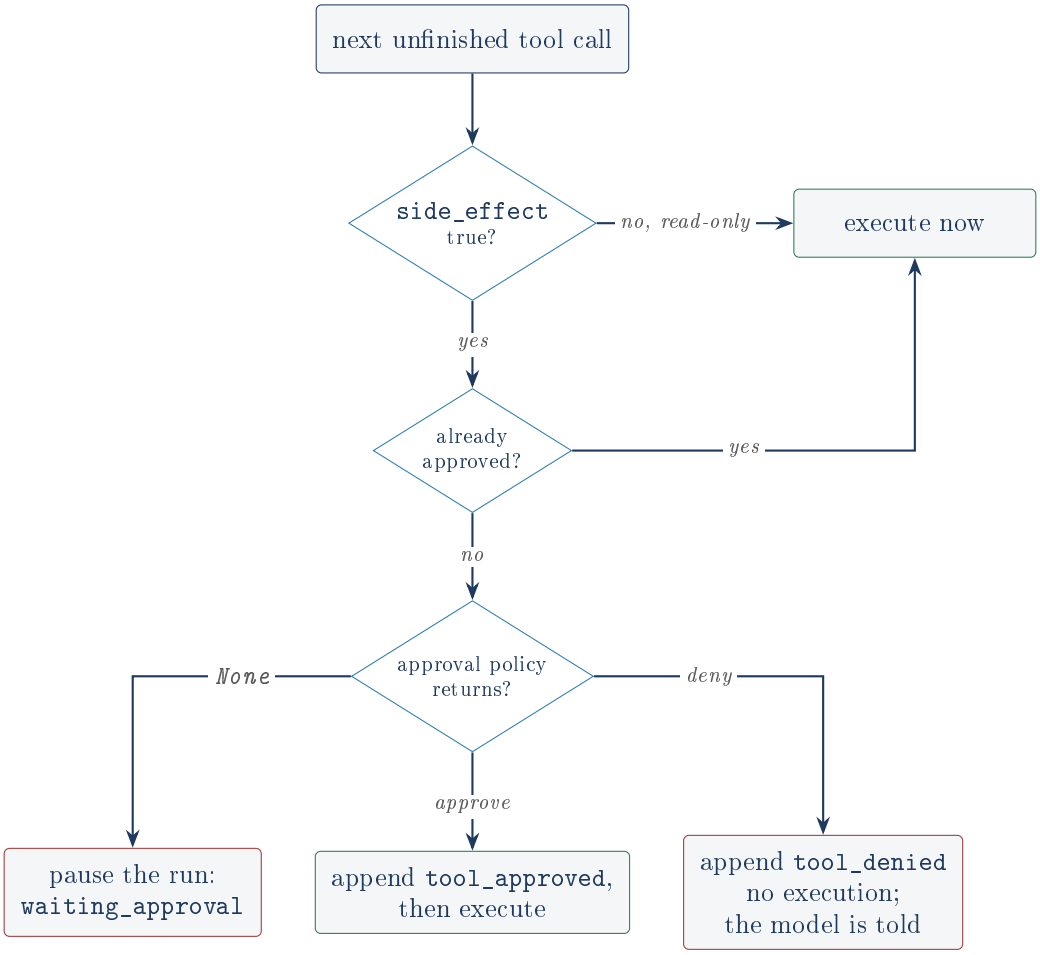

Figure 4: The gate. A read-only call never reaches the policy. `None` is the interesting return: it means "a human must decide", and the loop returns immediately without touching any later call.

An *approval policy* is just a function from a tool call to one of three answers: `("approve", who)`, `("deny", who, reason)`, or `None` meaning ask a human. That is the whole abstraction. The CLI's `--auto-approve` passes a function returning approve; the default passes no policy at all, which pauses; a scenario can pass one that denies specific tools by name.

The gate is safe because of ordering, not because of a lock

The pause is not a thread blocking on a condition variable, and there is no in-memory "pending approvals" queue. The run simply stops and the process can exit. The pending approval exists only as a fact about the log: a `tool_requested` event with `side_effect: true` and no

matching decision event after it. Any process, minutes or days later, can read the log and see it. That is why the CLI and the web UI can both serve the same approval queue without sharing anything but the database file.

`approve_call()` and `deny_call()` in `runtime.py` are the out-of-band entry points the CLI and web UI use. Both call `_assert_pending()` first, which re-projects the log and raises `ValueError` unless the call really is pending, so approving the same call twice, or approving a call that was already denied, is rejected rather than silently appending a contradictory event.

8 Idempotency and the crash window

Jargon: Idempotency

An operation is *idempotent* if doing it twice has the same effect as doing it once. Re-reading a file is naturally idempotent. Sending a reminder email is not: sending it twice annoys the customer twice. This project makes tool execution idempotent *per run* by remembering that it already ran.

`ToolExecutor.execute()` derives a key from `(run_id, seq of the tool_requested event)`:

```
def idempotency_key(run_id, requested_seq):
    return hashlib.sha256(f"{run_id}:{requested_seq}".encode()).hexdigest()[:32]
```

Both inputs are identical whether this is the first attempt or a resume after a crash, so the key is stable. That is the whole trick, and it is why the key is derived from the log rather than generated: a generated id would be lost in the crash it is meant to survive.

The write order is deliberate:

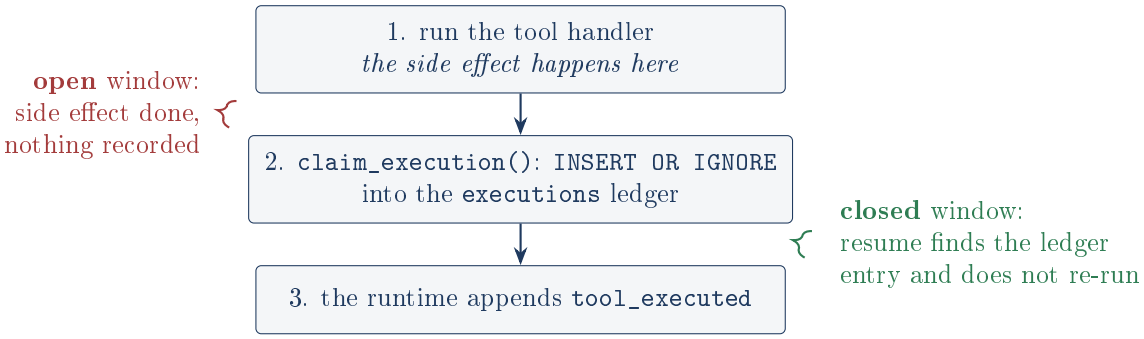

Figure 5: The execution order in `tools/executor.py`, and the two windows it creates. Only the lower one is closed.

If the process dies between step 2 and step 3, a subsequent resume calls `execute()` again with the same key, finds the row already in the ledger via `get_execution()`, and returns that recorded result without touching the handler. The runtime then appends `tool_executed` with `replayed: true`, so the log records that this execution did not actually re-run. `tests/test_resume.py` exercises exactly this and asserts a side-effect counter stays at 1.

The window between the handler and the ledger is open, and untested

The README says: "A crash between 'handler ran' and 'tool_executed appended' is where resume would naively re-execute. The executor writes the result to an `INSERT OR IGNORE` ledger ... before the runtime appends the event." That is true, and it closes most of the window. It does not close all of it.

Read `ToolExecutor.execute()` closely. The handler returns, and the ledger write happens on the very next line:

```
result = self._run_with_timeout(spec, args)   # the side effect happens here
self.store.claim_execution(key, run_id, result)   # ... and is recorded here
```

A crash between those two statements leaves the side effect done and nothing at all recorded, and a resumed run will run the handler a second time. The email goes out twice.
The test does not cover this. `CrashAfterExecute` in `tests/test_resume.py` wraps the executor and raises *after calling `self.inner.execute(...)` in full*, which includes the ledger write. So the test crashes in the closed window, which is why the assertion passes. Its docstring calls it "the worst possible crash point for duplicate side effects", and that is the one thing it is not: the genuinely worst point is one line earlier, inside `execute()`, and nothing tests it.
This is defensible, and worth being able to defend precisely rather than deny. The gap is inherent: you cannot atomically perform a real-world side effect and a local database write, and no amount of reordering fixes that. Shrinking the window is the best a local ledger can do. Genuinely closing it requires the side effect itself to carry the idempotency key so that the *remote* system deduplicates, which is what payment APIs do with an `Idempotency-Key` header. The honest claim is "the duplicate window is reduced from the whole tool execution to two adjacent statements", not "no duplicate side effects".

The executor also distinguishes two kinds of failure, which buys the retry policy for free:

- `ToolError` is an *expected*, deterministic refusal: a path outside the sandbox, a host not on the allowlist, a missing file, a cell out of range. Retrying cannot help, so it fails immediately.
- `ToolTimeout` (a subclass of `ToolError`, caught first) and any other exception are treated as transient and consume the retry budget with backoff.

Tool timeouts do not stop the tool (the README states this one)

`_run_with_timeout` submits the handler to a one-worker `ThreadPoolExecutor` and calls `future.result(timeout=...)`. When that times out the code calls `future.cancel()`, which for an *already running* task returns `False` and does nothing: Python cannot kill a running thread. The handler keeps running to completion in the background, invisible.
Three consequences the README names two of. A genuinely hung tool leaks a thread per attempt. A timed-out side-effecting tool may still complete its side effect after the runtime has recorded it as failed. And, not in the README: the `with ... ThreadPoolExecutor` block cannot exit until the abandoned worker finishes, because `__exit__` calls `shutdown(wait=True)`, so a hung tool hangs `execute()` itself rather than returning after `spec.timeout`. The timeout is honest about "how long do I wait for an answer" only when the tool eventually returns. Subprocess isolation is the real fix, and the README says so.

9 Deterministic replay

9.1 What deterministic replay means, and why an agent needs it

Jargon: Deterministic

A process is *deterministic* if the same inputs always produce the same outputs. Adding two numbers is deterministic. Reading the clock is not. Asking an LLM a question is emphatically not.

Jargon: Deterministic replay

Re-executing a program's logic using only what was recorded about a previous execution, with no live external calls, and checking that the result matches what was recorded. Every input the program would normally fetch from the outside world (the next model turn, a tool's result, a human's decision, the wall clock) is served instead from the recording.

Here is why this matters more for an agent than for ordinary software. Suppose run 4187 drafted an email to the wrong customer, and you want to know why.

You cannot just run it again. The model is nondeterministic, so the second run may behave perfectly and tell you nothing. It costs money each time you try. And running it again would draft *another* email, so your debugging tool has side effects. The usual approach of "add a print statement and re-run" is unavailable on all three counts.

Replay dissolves the problem. The log already contains every nondeterministic input the run received. Feed those recorded inputs back into the same loop and the run is now a pure function: it must produce the same events, every time, for free, with no email sent. You can step through it, and you can do it a year later.

Determinism is a budget you spend field by field

The reason replay works here is not a clever replay engine. It is that every event payload was designed so replay can recompute it. Any value that is not a pure function of the log is a future divergence: a wall clock reading, a random id, an attempt count that depends on real sleeps, a "request sent at" timestamp.

So the store takes an injectable clock, sleeps are injectable and become no-ops under replay, and `model_requested` records a *digest* of the messages rather than anything time-dependent. Each of those is a small design concession made specifically to keep replay exact. The comparator can then demand byte equality instead of some fuzzy "close enough" match, which is the difference between a real regression tripwire and a test that nobody trusts.

9.2 The four stand-ins

`replay_run()` builds four objects, each substituting for a real dependency, all reading from the same source event list:

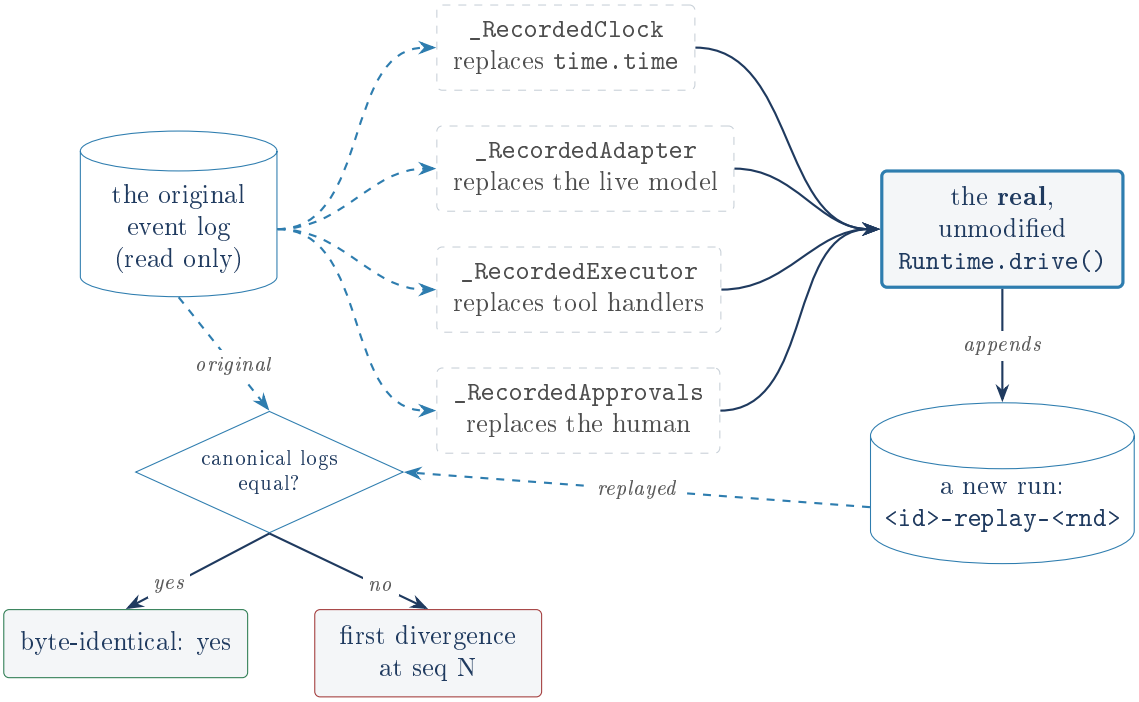

Figure 6: The replay path. Note what is *not* swapped: the runtime itself. That is the point. If the loop were reimplemented for replay, replay would prove nothing about the loop.

Stand-in	Replaces	Replays from
`_RecordedClock`	`time.time`	the exact `ts` of each original event, handed out in append order
`_RecordedAdapter`	the live `ModelAdapter`	each `model_error` or `model_responded` event in order, re-raising errors and malformed outputs exactly as before
`_RecordedExecutor`	`ToolExecutor`	the `tool_executed` or `tool_failed` outcome recorded for each `tool_requested` sequence number
`_RecordedApprovals`	the approval policy	the `tool_approved` or `tool_denied` decision recorded per call id

The clock deserves a second look, because it is the subtlest of the four. It is not a frozen time or a fake counter: it is a *queue* of the original timestamps, handed out one per `append()` call. This only works because appends happen in exactly the same order on replay. If they did not, timestamps would attach to the wrong events and the comparison would fail. So the recorded clock is not merely a convenience for making the diff pass; it is itself an assertion that the append order was reproduced.

9.3 Verified: it actually replays byte-identically

Running the real thing, not quoting the README:

```
$ agentrun replay 46bd97a45451
replayed 15/15 events into 46bd97a45451-replay-8765c6
byte-identical: yes

$ agentrun replay 46bd97a45451 --until 6
replayed 7/7 events into 46bd97a45451-replay-7c1c16
byte-identical: yes
state at event 6: status=running, model_calls=2, tools_executed=['read_file']
```

`--until <seq>` truncates the source log before replaying, giving a snapshot of the run's state at that exact point. Truncating can leave the loop asking for a model turn that was never recorded; rather than special-casing the loop, the stand-ins raise when exhausted and `replay_run()` treats that as the stop condition.

The truncation catch swallows every `RuntimeError`, not just exhaustion

`replay_run()` wraps `rt.start(...)` in `try: ... except RuntimeError: pass`. The intent, per the comment, is that a truncated log leaves the stand-ins exhausted and that is a normal stop.
But the catch is unconditional (it is not gated on `until is not None`) and `RuntimeError` is exactly what three unrelated failures raise: the clock's "replay produced more events than the original run", the adapter's "replay adapter exhausted", and `EventStore.append`'s "could not append event for run". A genuine bug in any of those is silently swallowed and reported as a normal stop.
The blast radius is limited, because the comparator runs afterwards and a truncated replay of a full log would come back as "not identical". So the failure surfaces, just mislabeled: you would be told the logs diverge at seq N when what actually happened is that the store could not write. Catching the specific exhaustion condition, or gating the catch on `until`, would cost one line.

9.4 Replay writes into the same database, and that has consequences

The replayed run is not a scratch object. `replay_run()` opens a second `EventStore` on *the same database file* and appends a complete new run under the id `<original>-replay-<random>`. Nothing is cleaned up afterwards.

This is visible in the repository itself. The tracked file `agentrun.db` contains 45 events across 3 run ids: one real run and two replays of it, matching the transcript in `docs/screenshots/run-output.txt` exactly. Every replay you ever run permanently doubles a run's storage and adds a row to `agentrun runs list`, which was verified:

```
$ agentrun runs list
46bd97a45451                finished   ops_assistant   15 events  $0.0032 simulated ...
46bd97a45451-replay-7c1c16  running    ops_assistant    7 events  $0.0005 simulated ...
46bd97a45451-replay-8765c6  finished   ops_assistant   15 events  $0.0032 simulated ...
```

That is untidy. The next finding is not.

A truncated replay can create a phantom approval that executes a real side effect. Verified.

This is the most serious thing in this document, and it was reproduced end to end.
A replay run is a real run in every way the rest of the system can detect. Its `run_created` event copies the original's `meta` verbatim (`meta=created.get("meta", {})`), including the real `workspace` path and `scenario_path`. The only thing marking it as a replay is a substring in its run id. Nothing in `projection.py`, `cli.py`, `web/app.py` or `bootstrap.py` looks for that substring.
Now replay with `--until` truncated exactly at a pending side-effecting call. The replayed run stops, correctly, at `waiting_approval`. It is now a non-terminal run with a pending approval sitting in the shared database:

```
$ agentrun replay d5ca78675af3 --until 8
replayed 9/9 events into d5ca78675af3-replay-6c2258
byte-identical: yes
state at event 8: status=waiting_approval, model_calls=2, tools_executed=['read_file']

```

```
$ agentrun approvals list
d5ca78675af3-replay-6c2258  call-2-0  draft_email  {'to': 'billing@acme.example', ...}
```

The operator's approval queue now lists a call that no live run is waiting on, indistinguishable from a real one. Approve it and `approvals approve` calls `resume_run()`, which rebuilds a runtime from that copied meta: a *real* executor, over the *real* workspace. The gate opens and the tool fires for real. Confirmed by deleting the outbox first, so that a reappearing file could only mean the handler genuinely ran:

```
$ rm -rf ws/outbox
$ agentrun approvals approve d5ca78675af3-replay-6c2258 call-2-0
run d5ca78675af3-replay-6c2258: finished
  final answer: Drafted a reminder email to Acme Corp for overdue invoice INV-1001.
$ ls ws/outbox/
Overdue_invoice_INV-1001.eml      <- written again, by a "replay"
```

The idempotency ledger cannot help: its key is `(run_id, seq)` and the replay's run id is different by construction, so the ledger sees a brand new call.

This directly undercuts the project's headline claim. "Replay: no network, no tool handlers, no key needed" is true *during* replay. It is not true afterwards, because a truncated replay leaves behind a live, resumable run that is no longer a replay and that no longer knows it ever was one.

Two changes would each independently close it, and both are small: put a `replay_of: <run_id>` field in the replayed `run_created` payload and have `resume_run`, `approvals list` and the web UI refuse or hide such runs; or replay into a throwaway database rather than the live one, which also fixes the storage growth and the polluted run list. The second is cleaner: a replay is a question, not a fact, and it has no business being written next to real runs at all.

Scope, stated fairly: a *full* replay ends in a terminal state and leaves no pending approval, so only `--until` triggers this, and `--until` is a debugging flag. The operator must also approve a call whose run id visibly contains `-replay-`. But the entire premise of an approval queue is that operators approve what it lists.

10 Failure injection

Jargon: Fault injection

Deliberately making a system behave as if something went wrong (a timeout, a malformed response, a server error) in a controlled test, to verify the system handles that failure the way it is supposed to, instead of only ever testing the happy path.

`faults.py` defines `FaultPlan`, a plain dataclass loadable straight from a scenario's YAML `faults:` block:

```
faults:
  model:
    - {at_call: 2, kind: malformed}      # or kind: adapter_500
  tools:
    read_file:
      - {at_attempt: 1, kind: exception, message: "disk error"}
      - {at_attempt: 2, kind: timeout}
```

`model_fault(call_number)` is consulted by `MockModelAdapter.complete()` on every invocation; `at_call` counts *every* adapter call, including ones that themselves fail. `tool_fault(tool_name, attempt)` is the executor's `fault_hook`, consulted before each attempt of a given tool. Both let a

scenario exercise "fails once, then a clean retry succeeds" without a real failing service. Unknown fault kinds are rejected at load time by `from_dict`.

The runtime's contract under faults: adapter errors retry with backoff then fail the run with a recorded cause; malformed output earns a corrective observation twice, then fails the run; tool failures surface to the model as ordinary error observations. Nothing raises out of `drive()`. `tests/test_faults.py` pins this down.

10.1 The two-counter bug, and why it is the interesting one

The README singles out a bug worth understanding, because the fix is what makes fault plans work at all. `MockModelAdapter` keeps two counters:

- `calls`: every invocation of `complete()`, including ones a fault plan turns into an injected error. This is what `at_call` matches against.
- `script_i`: how many script turns were actually *consumed*.

The first design used one counter for both. An injected 500 on call 1 silently consumed script turn 1, so every later turn was off by one and every scenario with a fault tested the wrong thing. Splitting them fixed it: injected faults advance `calls` but not `script_i`, so the script continues where it left off once the fault clears.

Verified directly, by driving the adapter with a fault plan and reading both counters: an injected malformed fault leaves `calls=1, script_i=0`. The script was not touched. Section 18 shows the other half of this story, which is not fixed.

11 Cost and latency accounting

Every `model_responded` carries a `usage` object: input and output token counts, latency in milliseconds, and a `simulated` boolean. `accounting.py`'s `account(events)` folds these into per-run totals using a hardcoded price table in USD per million tokens:

Model	Input $/1M tok	Output $/1M tok
`mock-1`	3.00	15.00
`claude-sonnet-4-5`	3.00	15.00
`gpt-4o`	2.50	10.00
(anything else)	3.00	15.00

The mock adapter fabricates deterministic but fake usage from message sizes, and `RunCosts.simulated` is true whenever any usage in the run came from it. The CLI and web UI both print "(simulated)" next to such a cost so it is never mistaken for a real bill. This was verified in the live output above: `cost:  $0.0021 (simulated)`.

Two small accounting inaccuracies

Neither is serious; both are the kind of thing worth knowing before someone asks.

`costs.model = model or costs.model` keeps the *last* model seen. A run whose adapter changed mid-flight (possible: `bootstrap` rebuilds the adapter from meta on every resume) reports one model name for the whole run. The per-event cost arithmetic is still correct, since each event is priced with its own model; only the reported label is lossy.

The default price silently applies to unknown models. A run against a model not in the table produces a confident dollar figure computed from Claude Sonnet's prices, with nothing in the output saying so. Given the README already concedes the table "will drift", a `price_source: default` marker in `RunCosts` would cost little.

12 Tools: registry, executor, built-ins

Jargon: JSON Schema

A specification language for describing the shape of JSON data: which fields exist, their types, which are required. Tool arguments are declared this way so badly formed arguments are rejected before a handler runs, and so the same declaration can be handed to an LLM API to tell it exactly what the tool expects. One declaration, two consumers.

`ToolSpec` captures, per tool: `name`, `description`, a JSON-schema `parameters` definition, the Python `handler`, the `side_effect` flag, a `timeout`, a `retries` count and a `backoff` base. `ToolRegistry` validates each schema at registration time with `jsonschema.Draft202012Validator.check_schema`, rejects duplicate names, and exposes `subset(names)`, used to restrict a run to only the tools its agent is allowed (Section 14).

Jargon: Sandbox

A restricted execution environment that limits what a program can access, to contain the effect of something going wrong or being misused. Here every filesystem tool resolves paths through `ToolContext.resolve()`, which calls `Path.resolve()` (collapsing any `..` segments and following symlinks) and then rejects the path unless it is the workspace root itself or has the workspace root among its parents. So a model cannot be talked into reading `../../etc/passwd`.

12.1 The eight built-in tools

Tool	What it does	Side effect?	Notes
`read_file`	read a UTF-8 text file	no	content truncated to 20,000 chars
`write_file`	write a UTF-8 text file	**yes**	creates parent dirs
`list_files`	recursively list files under a directory	no	capped at 500 names
`http_get`	GET a URL	no	host must be on an explicit allowlist, else `ToolError`; 15s timeout
`csv_update`	`append_row` or `set_cell` on a CSV	**yes**	out-of-range cell is a `ToolError`
`draft_email`	write an RFC 5322 `.eml` into the workspace `outbox/`	**yes**	never sends; filename sanitized from the subject
`calendar_add`	insert a row into a per-workspace SQLite `calendar.db`	**yes**	
`calendar_list`	read the calendar back, ordered by start time	no	

Jargon: Allowlist

A list of explicitly permitted values (here, hostnames) where everything not on the list is rejected by default. The opposite of a blocklist, and the safer default for anything reachable by a semi-autonomous actor: a blocklist fails open on the case you did not think of, an allowlist fails closed.

`http_get` is marked read-only, which is a judgment call, not a fact

`http_get` has `side_effect=False`, so it never reaches the approval gate. An HTTP GET is *conventionally* safe, but nothing enforces that: a GET can trigger anything the server chooses, and `follow_redirects=True` means the allowlist is checked against the URL the model asked for, not against wherever it ends up. An allowlisted host that redirects offsite is fetched without a second check.
In practice the allowlist defaults to empty (`AGENTRUN_HTTP_ALLOWLIST` unset blocks every host), so the exposure needs an operator to opt in. Worth naming anyway: this is the one tool whose safety rests on a convention about the wider world rather than on a check in this codebase.

`draft_email` silently overwrites drafts with the same subject

The `.eml` filename is derived only from the subject, sanitized and truncated to 60 characters. Two drafts to *different recipients* with the same subject, or two subjects agreeing in their first 60 characters, resolve to the same path and the second overwrites the first with no error. The tool's returned `draft_path` then names a file whose contents belong to a different email than the one the log says was drafted. The audit trail and the artifact disagree, which is a poor property for the one tool the approval gate exists to guard.

13 Model adapters

Jargon: Adapter

A class implementing one interface (`ModelAdapter.complete(messages, tools)` → `ModelTurn`) around a specific way of getting the next turn: a real API, or a scripted fake. The rest of the runtime only ever talks to this interface, never to a provider's request or response shape directly. That is what makes the mock adapter possible, and the mock adapter is what makes the whole project testable offline.

`adapters/base.py` defines the shared `ModelTurn` (text, tool calls, `Usage`, model name, with `is_final` true exactly when there are no tool calls) and the two exceptions the runtime catches by name: `AdapterError` (transport or provider failure, retryable) and `MalformedOutputError` (output that could not be parsed at all, carrying the raw text for diagnosis).

13.1 `MockModelAdapter`

`adapters/mock.py` plays back a *script*: an ordered list of turns from a scenario YAML, each either `tool_calls`, `final` text, or `malformed` raw garbage. Usage numbers are fabricated deterministically from message and output size and always carry `simulated: true`. Call ids are generated as `call-{calls}-{i}`, which is where the `call-2-0` in the transcripts comes from: the second adapter call, first tool call in it.

"Surfaced as a hard error" is not what happens when a script runs out

The comment above the script-exhausted branch says running past the end of a script "is a scenario bug, surfaced as a hard error rather than a guess". But the code raises `MalformedOutputError`, and `runtime.py` catches that by name and handles it gracefully: it records the run as having received malformed model output, feeds the model a corrective "your previous output could not be parsed" message, and after `max_malformed` tries fails the run with cause `malformed_model_output: mock script exhausted after N turns`.

So a scenario authoring mistake is reported as a *model behavior* problem. The cause string does carry the real reason, which saves it, but the run's shape (two corrective retries, then a malformed failure) is the shape of a misbehaving model, not of a too-short script. A distinct exception type that `drive()` does not catch would be a hard error; `MalformedOutputError` is by construction the one thing that cannot be.

13.2 `AnthropicAdapter` and `OpenAIAdapter`

Both implement request building and response parsing for their provider: `POST /v1/messages` (Anthropic Messages API) and `POST /v1/chat/completions` (OpenAI Chat Completions), keyed by `ANTHROPIC_API_KEY` and `OPENAI_API_KEY`, over `httpx`. Each translates the neutral message list into its provider's wire shape: Anthropic's `tool_use` and `tool_result` content blocks (where a tool result is carried on a `user` message) versus OpenAI's `tool_calls` array and a dedicated `tool` role. Both map HTTP ≥ 500 and other non-200 responses to `AdapterError`, and an unexpected response shape to `MalformedOutputError`.

Both take an injectable `client`, which is what makes them testable with no network: `tests/test_adapters.py` passes a fake `httpx.Client`.

Jargon: Contract test

A test that checks a piece of code produces the exact request shape a real API expects and correctly parses the exact response shape that API returns, without calling the real API. It verifies this side of the contract between two systems. It cannot verify the other side: if the provider's real API differs from what the test assumes, the test still passes.

The live adapters are unverified, and the README says so

Quoting the project's own scope note: "The Anthropic and OpenAI adapters are implemented against their current APIs and covered by offline contract tests (request building, response parsing, error mapping via injected HTTP transports), but **no live API calls were made here because no API keys were available**. Everything else, including the full test suite, the CLI and the web UI, runs and was verified entirely offline."
This document confirms the second half by running it (Section 21) and cannot confirm the first half either. The honest reading: the contract tests prove the adapters are self-consistent, not that they are correct against the real APIs. What would actually be caught by a live call and not by these tests: a required header, a renamed field, a changed enum value, an auth detail, a tool-schema constraint the provider enforces. That is a real gap, and the README is right to name it rather than imply coverage the tests do not give.

14 Agents and scenarios

`agents.py` defines three agents, each a fixed system prompt plus a restricted tool subset. The subset is enforced by `registry.subset()`, so a tool a agent is not granted is not merely discouraged by the prompt, it is not in the registry the adapter is shown, and a call to it would append `tool_failed` with "unknown tool".

Agent	Role and allowed tools
`ops_assistant`	Chases overdue invoices, drafts (never sends) reminder emails, keeps an invoice CSV current. Tools: `read_file`, `list_files`, `csv_update`, `draft_email`, `calendar_add`, `calendar_list`.
`data_entry`	Transcribes records into CSV sheets, verifying by reading back. Tools: `read_file`, `list_files`, `write_file`, `csv_update`.
`research_summarizer`	Fetches allowlisted pages and local notes, writes a sourced summary. Tools: `read_file`, `list_files`, `http_get`, `write_file`.

The repository ships 24 scenario YAML files, 8 per agent folder, counted directly from the tree and confirmed by the suite reporting 24 scenarios when run. The README's "Challenges" section notes this count was itself once wrong and was corrected by checking the files; it is right now.

15 Scenario regression testing

Jargon: Regression test

A test that exists specifically to catch something that used to work and now behaves differently. Here, the thing that changes is not the code: it is the model, or the prompt. That is the point.

What is actually being tested

This is the part most agent demos have no answer for. You cannot unit-test "the agent does the right thing", because the agent's behavior lives in a model you do not control and cannot pin. So this project tests the loop against a *recorded* behavior instead: a scenario names one or more behavior versions, each a full script of what the model would say, and asserts what the runtime does with it.
That reframes the question from "is the agent correct" (unanswerable) to "if the model behaves like *this*, does the runtime do the right thing, and does behavior `v2` differ from `v1`, and where exactly" (answerable, offline, in seconds, for free).

Each scenario YAML is validated against a strict JSON schema in `loader.py` with `additionalProperties: False` throughout, so a typo in a key is a load error rather than a silently ignored assertion. `load_pack` also rejects duplicate scenario names across the tree. A second, hand-written check enforces what JSON Schema expresses awkwardly: each behavior turn must have exactly one of `tool_calls`, `final` or `malformed`.

15.1 Assertions

`check_expectations()` evaluates every assertion present and never short-circuits, so a failing scenario reports everything wrong at once:

Assertion key	Checks
`status`	final run status
`tools_executed`	the exact, ordered list of tools that executed successfully
`tools_executed_contains`	these tools ran somewhere (order and extras ignored)
`tools_not_executed`	these tools never ran
`denied_tools`	these tools were denied at the gate
`final_contains` / `final_not_contains`	case-insensitive substring checks on the final answer
`failure_cause_contains`	substring of the recorded failure cause
`files_exist`	workspace-relative paths that must exist afterwards
`min_events`	minimum total event count
`no_side_effect_before_approval`	every executed side-effecting call has a `tool_approved` event at a *lower seq* than its `tool_executed`

The last one is the safety property of the whole project, expressed as an assertion over the log. Its own comment concedes it "always holds by construction; the assertion documents it per scenario". That is a fair description and a reasonable thing to assert anyway: it is a tripwire on the construction, and it costs nothing.

15.2 Verified: running the suite

```
$ agentrun test scenarios/ --behavior v1
behavior v1: 24/24 scenarios passed
  [PASS] batch-entry-with-listing (20 events, status=finished, cost=$0.0032 simulated)
  [PASS] create-new-sheet (10 events, status=finished, cost=$0.0012 simulated)
  ...

$ agentrun test scenarios/ --behavior v1 --compare v2
comparing v1 -> v2: v2 fails 3 of 24 scenarios (3 regressions vs v1)
  [REGRESSION] chase-overdue-acme
      first divergence at event 3: expected tool_requested:read_file,
                                       got tool_requested:draft_email
      v2 failure: tools_executed: expected ['read_file', 'draft_email'],
                                   got ['draft_email']
  [REGRESSION] transcribe-two-records
      first divergence at event 14: expected tool_requested:csv_update, got run_finished
  [REGRESSION] summarize-notes
      first divergence at event 8: expected tool_requested:write_file, got run_finished
```

The `v2` behaviors are deliberately degraded: `chase-overdue-acme`'s `v2` drafts the email *without reading the invoice first*, which is exactly the kind of regression a new model version might introduce and exactly the kind a pass/fail assertion alone would describe uselessly. The diff names the event.

The mechanism is deliberately unsophisticated and it is worth defending as such. `event_signature()` reduces each event to a `"type:tool-or-call-id"` string; `diff_results()` walks two signature lists in lock-step and reports the first index where they differ. No diffing library, no heuristics. For an ordered list of short strings, first-mismatch is the correct and complete answer. The README's claim that "a per-event type:detail signature is enough for useful regression diffs" is borne out by the output above, which is genuinely the most useful line in the whole tool.

16 The command line interface

`cli.py` exposes a single `agentrun` entry point registered via `pyproject.toml`'s `[project.scripts]`, built with **Click**, a library for building command line tools out of decorated functions.

Command	What it does
`agentrun run --scenario ...`	start a mock-adapter run scripted by a scenario; `--auto-approve` skips the gate
`agentrun resume <run-id>`	rebuild the runtime from `run_created` meta and drive forward
`agentrun replay <run-id> [--until N]`	deterministic replay; exits nonzero if not byte-identical
`agentrun runs list / show`	list runs with status and cost, or print one run's canonical log
`agentrun approvals list|approve|deny`	manage the approval queue; `deny --reason`
`agentrun test <pack> [--behavior --compare --html]`	the scenario suite
`agentrun serve [--host --port]`	the FastAPI web UI via `uvicorn`
`agentrun agents`	list agent definitions and their tools

Every command takes `--db` (default `agentrun.db`, overridable by `AGENTRUN_DB`). Both `replay` and `test` exit nonzero on failure, which is what makes them usable in a CI pipeline.

The default database path writes into the repository, and one is committed

`DEFAULT_DB` is the bare relative path `agentrun.db`, so every command run from the project root creates or extends `agentrun.db` *in the working tree*. `.gitignore` does not list it. Consequently `agentrun.db` is tracked in git: 36 KB holding 45 events across 3 run ids, one real run and two of its replays, exactly matching the demo transcript in `docs/screenshots/run-output.txt`. Following the README's own quickstart therefore dirties the repository, and `git status` reports a modified binary the user did not knowingly touch. The stored database is also not inert documentation: it contains a run whose `meta` points at an absolute workspace path from the author's machine.
Nothing here is dangerous, and the file is small. But an event store is generated output, and the fix is two lines: add `agentrun.db` to `.gitignore`, remove it from the index, and default `AGENTRUN_DB` somewhere outside the tree. If the committed database is meant as a demo fixture, that intent should be stated and it should be named like one.

17 The web UI

`web/app.py` builds a small **FastAPI** application (a Python web framework; here used to serve plain HTML rather than a JSON API) with three routes:

- `GET /`: a table of every run: id, agent, status badge, event count, cost.
- `GET /runs/{run_id}`: one run's status, cost and latency summary, any pending approvals rendered as a diff-style preview, and the full event timeline.
- `POST /runs/{run_id}/calls/{call_id}/approve` and `/deny`: record the decision, then call `resume_run()` *synchronously inside the request handler* before redirecting back.

All HTML is built by string concatenation with `html.escape()` applied to every interpolated value, including the model's own text. That matters more than it looks: the model's output is untrusted input, and it lands on a page an operator reads. The escaping is applied consistently in the routes reviewed.

17.1 Diff-style previews

`web/preview.py`'s `preview(tool, args, workspace)` renders what a *pending* call would change, before a human approves it. For `write_file` and `csv_update` it computes a real unified diff between the current file content and what the call would produce, by simulating the operation in memory. For `draft_email` and `calendar_add`, which have no natural "before" state, it renders a `+`-prefixed summary. Anything else falls back to pretty-printed JSON arguments.

Jargon: Unified diff

A compact standard format for showing the difference between two versions of a text file: unchanged lines plain, removed lines prefixed `-`, added lines prefixed `+`. What `git diff` prints.

This is the single most product-minded idea in the repository. An approval gate that shows an operator a blob of JSON arguments will be rubber-stamped within a day; one that shows the actual diff of the actual file is a control someone might really read.

The preview is a re-implementation of the tool, and can disagree with it

`preview.py` does not call the tool. It re-implements the tool's logic to predict the outcome: its own CSV load, its own `append_row` and `set_cell`. Two implementations of one behavior, with no test asserting they agree.

They already differ. `_csv_update` in `builtin.py` raises `ToolError` for an out-of-range cell; `preview.py`'s `set_cell` branch checks `if r < len(new_rows) and c < len(new_rows[r])` and, when false, silently changes nothing. So a proposed out-of-range `csv_update` previews as "(no textual change)", looks harmless, gets approved, and then fails. That is a benign direction to fail in. The dangerous direction is the same structure with the roles reversed: any future divergence where the preview under-states what the tool will do is a gate showing the operator something other than what they are approving. The fix is to have the tools expose a pure "compute the new content" function that both the handler and the preview call.

The gate has no authentication, and resumes inside the request handler

Two issues, one from the README and one not.

The README's own: "Approvals via the web UI resume the run synchronously inside the request handler. A slow tool blocks the HTTP worker for its whole timeout. A real deployment needs a worker process consuming a resume queue, with the UI only appending decision events." Correct, and Section 8's finding sharpens it: a *hung* tool does not merely block for its timeout, it blocks until the handler returns.

Not in the README: there is no authentication of any kind. No login, no token, no CSRF protection on the approve and deny POSTs. Anyone who can reach the port can approve any side effect, and any page an operator visits can forge a cross-site POST to it. The default bind is `127.0.0.1`, which is the mitigation and a real one, but `--host` is offered with no warning attached. For most CRUD apps this would be ordinary prototype scope. Here the approval gate *is* the security control the project is built around, so "the control has no access control" is worth stating plainly rather than leaving for someone else to notice.

18 Bootstrap, resume, and a real off-by-one

`bootstrap.py` exists so that "build me a working `Runtime` for this run" is written once and shared by the CLI and the web UI. Its key idea, from its own docstring: run metadata (workspace path, adapter kind, scenario file, behavior version) lives in the `run_created` event, so any process with

the database can rebuild the exact runtime needed to continue a run. No session state has to survive a process restart.

`resume_run(store, run_id, approval_policy)`:

1. Project the log; if the run is terminal, return immediately.
2. Count `model_responded` events that are *not* malformed. This is taken to be how many script turns the mock adapter consumed, and `script_i` is repositioned to it.
3. Separately count every `model_responded` *or* `model_error` event. This is how many times `complete()` was called in total, and repositions `calls` so that fault matching on `at_call` stays aligned.
4. Call `drive(run_id)`, which picks up where the fixed point left off.

Step 3 is the fix the README describes. Step 2 is wrong.

Verified bug: resume rewinds the script by one after a scripted malformed turn

The README concedes a nearby issue: "a run resumed in the middle of an injected model-fault sequence could realign fault triggers off by one." The actual defect is a different one, in the other counter, and it is demonstrable.

Read `MockModelAdapter.complete()` carefully. The script index advances *before* the turn's content is inspected:

```
step = self.script[self.script_i]
self.script_i += 1              # <- advances here, unconditionally
input_text = json.dumps(messages, sort_keys=True, default=str)

if "malformed" in step:
    raise MalformedOutputError("scripted malformed output", raw=step["malformed"])
```

So a *scripted* `malformed:` turn does consume a script slot. (An *injected* malformed fault does not, because the fault plan is consulted earlier and raises before this block. That asymmetry is deliberate and correct.) But that scripted turn is recorded as `model_responded` with `malformed: true`, and `resume_run`'s step 2 counts only `model_responded` events that are **not** malformed. It therefore does not count it.

Result: resuming a run whose script contained a scripted malformed turn rewinds `script_i` by one per such turn, and the malformed turn is served to the model again. Confirmed by driving the adapter directly:

```
scripted malformed: calls=1 script_i=1   -> consumed a script turn: True
injected malformed: calls=1 script_i=0   -> consumed a script turn: False

resume_run would set script_i = 0
the true position after that turn is  = 1
MISMATCH: resume rewinds the script by 1 turn; the malformed turn is served again.
```

Two shipped scenarios contain scripted malformed turns:

- `scenarios/data_entry/malformed_then_recover.yaml`
- `scenarios/research/malformed_output_fails_run.yaml`

Neither is ever resumed by the test suite, and the scenario runner drives runs start to finish in one process, so nothing exercises the combination. That is why 93 tests pass over a real bug: the bug lives precisely in the gap between two features that are each tested alone.

Consequence if hit: the resumed run re-raises the malformed output, burns a second entry against `max_malformed` (default 2), and can fail a run with `malformed_model_output` that would have completed had it never been interrupted. The fix the README already proposes is the right one and fixes both this and the issue it does concede: record the adapter cursor in

the `checkpoint` payload instead of inferring it. The cursor is the one piece of state that is genuinely not a function of the log, which is exactly why inferring it is where the event-sourced design springs a leak.

The honest generalization, and the best thing to say about it in an interview

The project's thesis is "all state is a function of the log". This bug is what happens at the one place where that is not quite true.

The mock adapter's script position is real state living outside the log, and `resume_run` tries to reconstruct it by inference. The inference is a heuristic ("one script turn per non-malformed response") that happens to be wrong for one case. Every other feature in this project is exact because it reads the log; this one is approximate because it guesses at something the log does not record.

That is not an argument against event sourcing. It is the strongest argument *for* it in the whole codebase: the single component that opted out of the discipline is the single component with a latent bug. And it is confined to the mock adapter, which is test infrastructure, so no production path is affected.

19 Package map

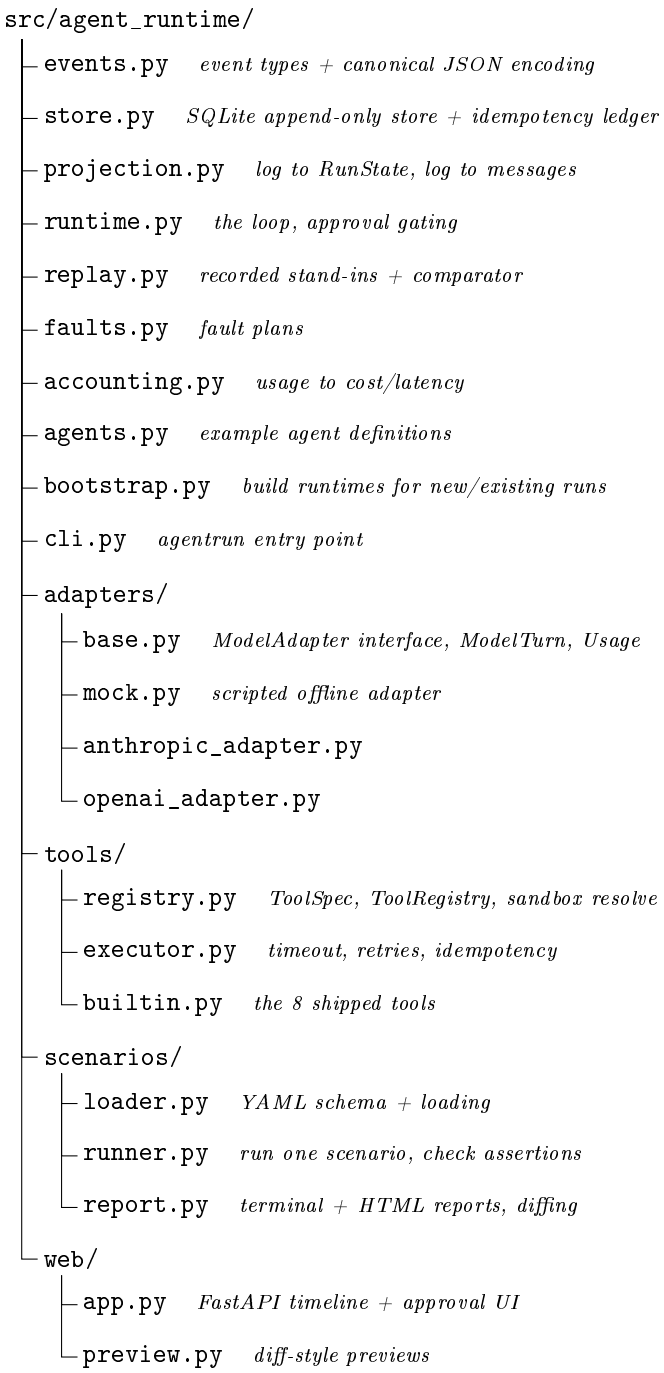

Figure 7: The package. Alongside it: `scenarios/` (24 YAML files in three folders), `tests/` (13 modules), `docs/`, and the standard `README.md`, `LICENSE` (MIT), `pyproject.toml`, `requirements.txt`, `.env.example`.

The dependency direction is worth noting: `events.py` imports nothing from the package, `store.py` imports only `events.py`, `projection.py` imports only `events.py`. The log and its interpretation sit at the bottom and know nothing about runtimes, adapters or tools. Everything else depends inward on them. That is why the claim "the projection is the single interpreter" is structurally enforced rather than a convention: no other module is positioned to reinterpret a log.

20 Dependencies

Package	Role
`click`	builds the `agentrun` CLI
`PyYAML`	parses scenario files; `yaml.safe_load` is used, not `load`
`jsonschema`	validates tool arguments and scenario structure
`httpx`	HTTP client for the live adapters and `http_get`
`fastapi` + `uvicorn`	the web UI and its ASGI server
`python-multipart`	required by FastAPI to parse the deny form's `reason` field
`pytest` (dev extra)	the test suite

Seven runtime dependencies for this much functionality is restrained, and two absences are deliberate and defensible: no ORM (the store is 120 lines of SQL), and no templating engine (the HTML is string concatenation with escaping). Both are the right call at this size.

A real bug the project's own history records

From the README: "The first full-suite run failed with 5 errors in the web tests only: FastAPI's `Form` parsing needs `python-multipart`, which nothing else imports, so every non-web test passed and the gap only surfaced when the deny-with-reason endpoint was exercised through `TestClient`. Pinned it as a real dependency rather than a test extra since the deny form is a runtime feature."
This is confirmed in `pyproject.toml`, where `python-multipart` sits in `dependencies`, not under `dev`. The reasoning is right: the deny form is a shipped feature, so its parser is a runtime dependency, not a test one.

21 Testing: what was actually run

The README claims "93 tests, all offline" and "no live API calls were made". Both were checked rather than taken on trust.

21.1 The count

```
$ ls tests/*.py | wc -l                 ->  13
$ cat tests/*.py | wc -l                ->  1357
$ grep -c "^def test_" tests/*.py | ...  ->  93
$ pytest --collect-only -q              ->  93 tests collected
```

93 test functions, 93 collected. There is no `parametrize` anywhere in `tests/`, so the collected count is not inflated by parametrization: 93 functions are 93 tests. The README's number is exact.

21.2 The run

```
$ pytest -q
.....................................................................  [ 77%]
....................                                                   [100%]
93 passed, 1 warning in 46.80s
```

The single warning is a `StarletteDeprecationWarning` from FastAPI's `TestClient` about `httpx`, raised inside the installed library, not by this project's code.

21.3 The "no live API calls" claim, tested adversarially

A passing offline test suite does not by itself prove no network call was made: a call could succeed silently, or fail and be swallowed by a retry. So the suite was re-run with a guard that monkey-patches `socket.socket.connect`, `socket.create_connection` and `socket.getaddrinfo` to raise on any non-loopback address, turning any real network access into a hard test failure:

```
$ pytest -q -p noweb
NETWORK GUARD: zero outbound connections or DNS lookups attempted.
93 passed, 1 warning in 55.01s
```

Not one outbound connection, and not even a DNS lookup. The claim is stronger than the README states it: the suite does not merely avoid *calling* the providers, it never resolves their hostnames. This is a verified, defensible claim.

Why this is architecture, not luck

Nothing in the test suite is skipped for lack of a key, and nothing is mocked at the last moment with a patching library. The adapters take an injectable `client`, the runtime takes an injectable adapter, the store takes an injectable clock, and the executor takes an injectable sleep and fault hook. The offline property falls out of the constructor signatures.
That is also the honest limit of it: the same design that makes 93 tests run with no network is the reason none of them can tell you the live adapters work.

21.4 Beyond the suite

The CLI was driven end to end for this document: a scenario run pausing at the gate, a CLI approval resuming it to `finished`, a full replay reporting `byte-identical: yes`, a `--until` replay reporting mid-run state, the 24 scenario suite passing 24/24 on `v1`, and the `v1` against `v2` diff reporting 3 regressions with first-divergence event numbers. Every transcript quoted in this document is real output from those runs. The two findings in Sections 9.4 and 18 were found that way.

Test coverage has a shape, and the gaps are where the bugs are

The suite is genuinely good: it tests the crash path, the fault paths, the replay comparator, denials, and the web endpoints. But two of the three defects found while writing this document sit in the seams between tested features, not inside any of them.
The scripted-malformed resume bug needs *scripted malformed* plus *resume*. Both are tested; the combination is not. The phantom-approval issue needs *replay –until* plus *the approval queue* plus *resume*. All three are tested; the combination is not. `test_replay.py` asserts replays are byte-identical but never asks what the replayed run *is* afterwards or what else can now see it.
This is the normal shape of a suite written feature by feature, and it is worth being able to name: the tests check that each feature works, not that the features cannot interact badly. The event log makes the cross-feature test easy to write, since every assertion is just a query over events. It has not been written yet.

22 Honest findings, collected

Everything below is stated in full, with evidence, at the section given. Ordered by how much it would hurt to hear from an interviewer first.

	Finding	Where	Status
1	A truncated replay leaves a phantom approval that, if approved, executes a real side effect in the real workspace	Sec. 9.4	**Found here.** Reproduced end to end
2	Resume rewinds the mock script by one after a scripted malformed turn	Sec. 18	**Found here.** Reproduced. README concedes a different, nearby case
3	The crash window between the handler and the ledger is open and untested; the test crashes in the closed window	Sec. 8	**Found here.** Inherent limit; claim should be narrowed
4	The web approval gate has no authentication or CSRF protection	Sec. 17	**Found here.** Mitigated by the localhost default
5	The preview re-implements tool logic and already disagrees with it on out-of-range cells	Sec. 17.1	**Found here.** Benign direction, bad structure
6	`agentrun.db` is committed; the default DB path writes into the repo	Sec. 16	**Found here.** Cleanliness
7	`except RuntimeError: pass` in replay swallows genuine store and adapter errors	Sec. 9.3	**Found here.** Mislabels, does not hide
8	A too-short mock script is reported as malformed model output, not as a scenario bug	Sec. 13.1	**Found here.** Comment overstates
9	`draft_email` overwrites drafts sharing a subject	Sec. 12.1	**Found here.**
10	`http_get` is ungated and its allowlist is not re-checked after redirects	Sec. 12.1	**Found here.** Allowlist defaults empty
11	Tool timeouts abandon the thread; a hung tool also hangs `execute()`	Sec. 8	README states the leak; the hang is found here
12	Live adapters never exercised against the real APIs	Sec. 13.2	README states it
13	Web approvals resume synchronously in the request handler	Sec. 17	README states it
14	Event payloads carry no schema version	below	README states it
15	The price table is hardcoded and will drift; unknown models silently get the default	Sec. 11	README states the drift

On finding 14, which has no section of its own because there is nothing in the code to point at: that is the point. Every event payload is written and read by whatever version of the code happens to be running. Replay detects that a change altered the loop's output, which the README fairly calls "a tripwire, not a migration story". The moment a payload shape changes, every old log becomes unreplayable with no way to tell which version wrote it. For a project whose entire premise is that the log is the durable asset, an unversioned log is the one structural gap that most deserves fixing first, and it is cheap now and expensive later.

The fair summary

Every finding above is a defect in the operational shell, and not one is a defect in the central idea. The event log, the projection as sole interpreter, the gate as an ordering property rather than a lock, and byte-identical replay all hold up under examination and under execution. The thesis is sound and the thesis is demonstrated: 93 tests pass with provably zero network, replay is byte-identical on real runs, and the regression diff does the thing it claims.

What the findings show is narrower and more interesting: the seams. The phantom approval happens because a replay is written into the same table as a real run and is distinguished only by a naming convention. The resume bug happens because one piece of state was inferred rather than recorded. Both are cases of the project not taking its own discipline quite far

enough, which is a much better thing to have to defend than a design that does not work.

23 Glossary

Adapter A class implementing a fixed interface around a specific model API or a fake.

Agent A program that uses an LLM in a loop to decide and take actions via tools.

Allowlist A set of explicitly permitted values; everything else is rejected.

Approval gate The point where a side-effecting call pauses for a human decision.

Backoff Waiting longer between each successive retry.

Canonical encoding A deterministic serialization where equal data always produces an identical string.

Checkpoint A periodic event recording loop progress; for readability, not correctness.

Contract test A test verifying request and response shape against an API's spec without calling it.

Deterministic Same inputs, same outputs, every time.

Digest / hash A short fixed-length fingerprint of a larger piece of data.

Event sourcing Storing every state change as an immutable ordered log instead of overwriting state.

Fault injection Deliberately simulating failures to test that they are handled.

Fixed point iteration A loop that recomputes its next step purely from its own last output.

Idempotency Doing an operation twice has the same effect as doing it once.

JSON Schema A specification language describing the required shape of JSON data.

LLM Large language model: a model that generates text or structured output from a prompt.

Primary key A column set guaranteed unique per row, used here to enforce append-only ordering.

Projection Computing derived state from an event log.

Regression test A test proving something that used to work still works after a change.

Replay Re-driving logic from only recorded data, with no live calls, checking the result matches.

Sandbox A restricted environment limiting what a program can access.

Side effect An action with a lasting, observable consequence outside the program.

SQLite A serverless relational database engine whose whole database is one file.

Unified diff A compact standard format for showing line-level differences between two texts.